

saves its state of execution via a context switch, and begin execution of an interrupt handler. Software interrupts are usually implemented as instructions in the instruction set, which cause a context switch to the interrupt handler, similar to a hardware interrupt.

Hardware interrupts comprise two major classes:

- Non-Maskable interrupts (NMI):** This is caused by some serious hardware problem like a memory error. Such an interrupt cannot be ignored or suppressed. This is quite obvious, since incorrect or malfunctioning hardware must be rectified before any other processing can be done reliably. This kind of interrupt cannot be delayed or rejected.
- Maskable interrupts:** This is a hardware interrupt that may be delayed, or rejected by setting a bit in an interrupt mask register's (IMR) bit-mask.

When an interrupt occurs

An interrupt causes the CPU to stop executing the current program, and start executing a special piece of code, called an interrupt handler, or interrupt service routine (ISR). The ISR does some work, and then resumes the interrupted program. Each interrupt has its own Interrupt Service Routine, which is looked up via the Interrupt Vector Table.

Interrupt Type	Address of ISR
INT 1	Address (ISR 1)
INT 2	Address (ISR 2)
INT 3	Address (ISR 3)
INT 4	Address (ISR 4)
:	:
:	:
INT n	Address (ISR n)

Figure 7: An example of an interrupt vector table

Interrupt vector table

This is a table of interrupt vectors, which associate an interrupt handler with an interrupt request. Interrupt vector tables contain the memory addresses of interrupt handlers. When an interrupt is generated, the processor saves its execution state via a context switch, and begins execution of the interrupt handler at the interrupt vector.

In my next article I will discuss how the context switch is carried out during an interrupt, and what we need to consider while designing an Interrupt Service Routine. 

Glossary

- Context switch:** A context switch is the computing process of storing and restoring the state of a CPU (the context) such that multiple processes can share a single CPU's resources.
- Interrupt handler:** This is also known as an Interrupt Service Routine (ISR). It is a subroutine whose execution is triggered by the reception of an interrupt.

References and further reading

- http://www.atarimagazines.com/compute/issue149/60_interrupts_made_easy.php
- <http://www.delorie.com/djgpp/doc/ug/interrupts/inthandlers1.html>

By: Deepti Sharma

The author is a senior software engineer with a considerable amount of experience in the design, development and quality validation of System Software Tools (compilers/assemblers/device drivers) for embedded systems. She has also been a mentor/trainer for compiler development, following good practices and maintaining quality. She loves writing for LFY. She can be reached at acmet@gmail.com.

A Computer Magazine

that does not talk about Music, Games,
Photo albums—**Only Business**



Read BenefitIT: A Business Minded Computer Magazine

www.benefitmag.com